```
"""Validate measurement CSV and export grouped descriptive statistics."""
import argparse
import csv
import json
import math
from pathlib import Path
def read_measurements(path):
    records = []
    with path.open(encoding="utf-8-sig", newline="") as stream:
        reader = csv.DictReader(stream)
        if not {"group", "value"} <= set(reader.fieldnames or []):
            raise ValueError("CSV must contain group and value columns")
        for line, row in enumerate(reader, 2):
            group = row["group"].strip()
            if not group:
                raise ValueError(f"Missing group at line {line}")
            try:
                value = float(row["value"])
            except (TypeError, ValueError) as exc:
                raise ValueError(f"Invalid value at line {line}") from exc
            if not math.isfinite(value):
                raise ValueError(f"Non-finite value at line {line}")
            records.append((group, value))
    if not records:
        raise ValueError("CSV contains no measurements")
    return records
def summarize(records):
    groups = {}
    for group, value in records:
        groups.setdefault(group, []).append(value)
    results = []
    for group, values in sorted(groups.items()):
        total = math.fsum(values)
        results.append({
            "group": group,
            "count": len(values),
            "sum": total,
            "mean": total / len(values),
            "minimum": min(values),
            "maximum": max(values),
        })
    return results
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("input", type=Path)
    parser.add_argument("--output", required=True, type=Path)
    args = parser.parse_args()
    try:
        records = read_measurements(args.input)
        result = {"record_count": len(records), "groups": summarize(records)}
```

```
    except (OSError, ValueError) as exc:
        parser.exit(2, str(exc) + "\n")
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(
        json.dumps(result, ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )
    print(f"Validated {len(records)} records; wrote {args.output.name}")
if __name__ == "__main__":
    main()
```